Problem 1 – Smoothing ReLU (30 Points)

Consider a feed-forward neural network $f : \mathbb{R}^d \to \mathbb{R}$ with one hidden layer, structured as a multi-layer perceptron (MLP) with ReLU activations. The network consists of a single hidden layer with weights and biases denoted by $W \in \mathbb{R}^{m\times d}$, $b \in \mathbb{R}^m$, and output layer weights $v \in \mathbb{R}^m$. For an input $x \in \mathbb{R}^d$, the network output is given by:

$$f(x) = v^T \phi(Wx + b),$$

where ϕ denotes the ReLU activation function, applied element-wise, defined by:

$$\phi(z) = \max(0, z).$$

Suppose there exists a target function $g : \mathbb{R}^d \to \mathbb{R}$ and a set of training data $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ where $y_i = g(x_i) + e_i$ and the noise term e_i satisfies $|e_i| \leq E$ for all $i = 1, \ldots n$.

Suppose the resultant network after training f^* with weights (v^*, W^*, b^*) satisfies

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} |f^*(x_i) - y_i|^2} \leq \epsilon.$$

(a) Show that there exists a function $h : \mathbb{R} \to \mathbb{R}$ which is independent of ϵ and only depends on E such that

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} |f^*(x_i) - g(x_i)|^2} \leq \epsilon + h(E).$$

(b) Let W_j and b_j denote the j-th row and coordinate of W and b, respectively. Is $\phi((W_j^*)^\top x + b_j^*)$ Lipschitz? why?

(c) Now, consider the process of smoothing the non-differentiable ReLU activations via randomized smoothing. Specifically, we modify the network by convolving each ReLU activation with a Gaussian probability density function with mean zero and variance σ^2.

Let the resulting smoothed function be denoted by f_σ^*. Note that we do not re-train the weights. Establish a valid upper bound on the following expression:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} |f^*(x_i) - f_\sigma^*(x_i)|^2},$$

in terms of (potentially a subset of) $\|v^*\|_2$, $\|W^*\|_F$, $\|b^*\|_2$, n, d, m, E, and σ.

(d) Using the results of part a and part c, find an upperbound on σ and an upperbound on E that are sufficient to ensure

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n} |f_\sigma^*(x_i) - g(x_i)|^2} \leq 2\epsilon.$$

Problem 2 – Learning Performance of a Simple DL Model (40 Points)

Consider the function $h : \mathbb{R}^d \to \mathbb{R}$ such that $h(x) = \int_u \alpha(u)\sigma(\langle u, x\rangle)du$, where $\alpha : \mathbb{R}^d \to \mathbb{R}$ and $\sigma : \mathbb{R} \to \mathbb{R}$ is the sigmoid activation function, i.e., $\sigma(z) = \frac{1}{1+e^{-z}}$. Assume we sample a data set $\mathcal{D} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$, where $x_i \overset{\text{i.i.d.}}{\sim} p_x$ for some probability density p_x, and $y_i = h(x_i)$ for all $i = 1, \ldots, n$.

(a) Assume we construct a randomized function with the goal of estimating h. Our construction takes the following form

$$\tilde{h}_u(x) = \frac{1}{N}\sum_{j=1}^{N} \tilde{w}(u_j)\,\sigma(\langle u_j, x\rangle) \tag{5}$$

where $u_j \overset{i.i.d.}{\sim} p_u$ for some probability density p_u and $\tilde{w} : \mathbb{R}^d \to \mathbb{R}$. note that $\tilde{h}_u$ is a randomized estimator where the source of randomness is in the selection of $u_1, \ldots, u_N$. Find $\tilde{w}(\cdot)$ as a function of α and p_u such that $\tilde{h}_u$ is an unbiased estimator of h with bounded variance, i.e.,

$$\mathbb{E}_{u_1,\ldots,u_N}[\tilde{h}(x)] = h(x), \quad \text{and} \quad \mathbb{E}_{u_1,\ldots,u_N}\left[\left|\tilde{h}_u(x) - h(x)\right|\right] \le \frac{A}{\sqrt{N}}, \quad \forall x \in \mathbb{R}^d \tag{6}$$

where $|\tilde{w}(u)| \le A$.

(b) Let $f_u(x)$ be a randomized estimator of $h(x)$ where the subscript u denotes the source of randomness in f_u. Define

$$\mathcal{L}(f_u) := \mathbb{E}_{u,x\sim p_x}|f_u(x) - y| \tag{7}$$

where $y = h(x)$. Using this, show that $\mathcal{L}\left(\tilde{h}_u\right) \le \frac{A}{\sqrt{N}}$.

(c) We now consider estimating h using an artificial neural network (ANN). Let $f(x) = w^\top \sigma(Ux)$, where $w \in \mathbb{R}^N, U \in \mathbb{R}^{N\times d}$, and σ is the sigma activation function applied coordinate wise to its argument. What is the architecture of this ANN? Describe all layers and their associated parameters in details.

(d) Using $\mathcal{D}$ we form an optimization problem to train the ANN from part 3. Here is what we will do: we will initialize w arbitrarily, however we will initialize each row of U (denoted by $u_j \in \mathbb{R}^d$,
$j = 1, \ldots, N$) using p_u from part 1 and freeze them throughout training. That is, using $\mathcal{D}$, will only optimize for w. In particular, we will form the following problem

$$\begin{aligned} w^* = &\arg\min_w \frac{1}{n}\sum_{i=1}^{n}\left|y_i - w^\top\sigma(Ux_i)\right| \\ &\text{s.t. } \|w\|_p \le B \end{aligned} \tag{8}$$

where $\|\cdot\|_p$ denotes the ℓ_p-norm. Find p and B such that $\tilde{h}_u$ is a feasible solution to this optimization problem.

(e) Let $h_u^* = {w^*}^\top \sigma(Ux)$ be the ANN found using the training problem in part d. Show that $\mathcal{L}(h_u^*) \leq \frac{A}{\sqrt{N}}$.

Problem 3 – A DL problem using DL packages (30 Points)

In this problem we focus on the task of training a deep learning model for image classification for the Fashion-MNIST data set.

We will implement an neural network with the backbone of ResNet-18 as shown in Figure 1. (ResNet model was introduced in the paper Deep Residual Learning for Image Recognition, and released in this repository).

The first two layers of ResNet are the 7×7 convolutional layer with 64 output channels and a stride of 2 is followed by the 3×3 max-pooling layer with a stride of 2. The batch normalization layer added after each convolutional layer in ResNet. There are 4 convolutional layers in each module (excluding the 1×1 convolutional layer). Together with the first 7×7 convolutional layer and the final fully connected layer, there are 18 layers in total. Therefore, this model is commonly known as ResNet-18. By configuring different numbers of channels and residual blocks in the module, we can create different ResNet models, such as the deeper 152-layer ResNet-152. We will use Adam as our optimizer instead of SGD. (Adam optimizer was introduced in the paper Adam: A Method for Stochastic Optimization, with additional description in Adam docs by PyTorch). According to this paper, the method is computationally efficient, has little memory requirement, invariant to diagonal rescaling of gradients, and is well suited for problems that are large in terms of data/parameters. Write a Python program to solve the training problem with below settings using ResNet-18, Adam for 10 epochs and a batch size of 256. Plot the training and test accuracy curves versus epochs and report the learning rate η, other parameters of Adam optimizer, and the regularization hyper-parameter λ you used. Submit your code as a PDF file combined with the rest of your submission.

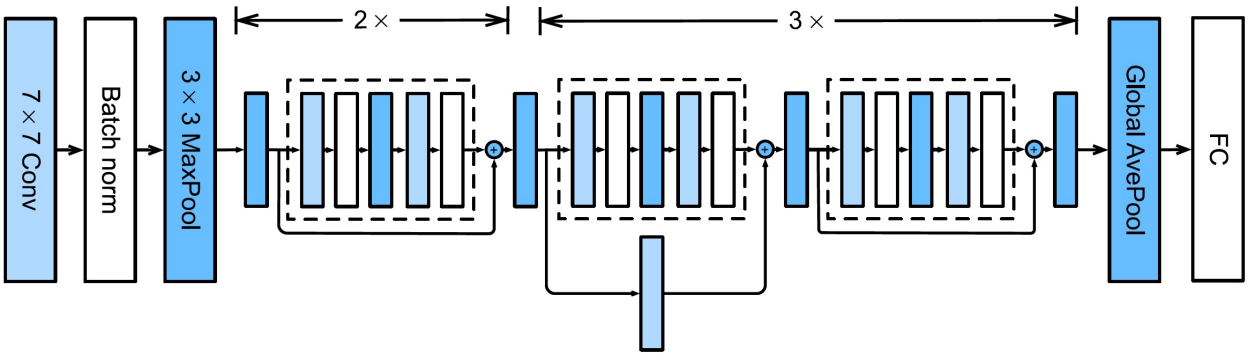


Figure 1: The ResNet-18 architecture

a) Try adding ℓ_2 regularization with a specific value of λ of your own choosing.

b) Try applying weight decay using the same value of λ from part a.

c) Now, use AdamW instead of Adam with weight decay using the same value of λ from part a.

d) Apply early stopping to the results of part a, b, and c and report how this will change the training and test performance.

Note: We are allowed to use DL packages, such as PyTorch, to implement ResNet-18, Adam and AdamW optimizers, ℓ_2 regularization, weight decay, and early stopping strategies.

Some example codes using ResNet-18: Kaggle Dataset Notebooks, pre-trained ResNet, ResNet-18 docs, and ResNet implementation by PyTorch. Example code using early stopping: Early Stopping for PyTorch.